Predicting Text from Intracranial Neural Signals

Johnny You
UCLA
johnnyou1873@ucla.edu

Leo Zhang
UCLA
leozhang1216@ucla.edu

Yueran Dong
UCLA
yueran02@ucla.edu

Pinhao Hong
UCLA
lucianhong124@ucla.edu

Abstract

We explore multiple independent approaches to improve the real-time phoneme decoding of intracranial neural signals using the Brain-to-Text Benchmark '24 dataset. We were provided a GRU model with day-specific input layers to serve as a benchmark for evaluating model performance. This model achieves a validation phoneme error rate (PER) of approximately 0.22 in a streaming setup. Each team member attempts to improve GRU performance without significantly increasing model size through a variety of architectural, optimization, and encoding modifications. We find that diphoneme encoding, with extended training with the AdamW optimizer and CosineAnnealingLR, decreases PER to approximately 0.179. A lightweight 7-million-parameter phoneme language model applied to the GRU output does not significantly improve performance but yields qualitative improvements in cohesion of the final decoding. Other architectural and dataloader changes which do not produce significant improvements in performance but were explored include the insertion of LayerNorm before the GRU, adding a one-layer Transformer encoder, and applying stronger noise and feature-masking augmentations. Together, our independent investigations highlight several promising directions for improving intracortical speech decoding performance while preserving real-time streaming constraints. The public git repository with the code can be found here: https://github.com/johnnyou1873/c143a-project.

1 Introduction

1.1 Background and Motivation

Speech neuroprosthetics aim to restore communication for individuals who have lost the ability to speak due to conditions such as ALS, and recent work has shown that intracortical brain–computer interfaces (BCIs) can decode attempted speech from motor cortical activity with high temporal resolution [**willett2021BCI**, **willett2023high**]. The Brain-to-Text '24 benchmark evaluates this capability by requiring models to decode phoneme sequences from neural recordings without explicit alignment. Its baseline system—a unidirectional GRU [**cho2014gru**] trained with Connectionist Temporal Classification (CTC) loss [**graves2006ctc**]—achieves a validation PER of approximately 0.22 and serves as the reference point for evaluating model improvements under real-time constraints.

1.2 Dataset

The Brain-to-Text '24 benchmark dataset contains microelectrode array recordings from the ventral premotor cortex (area 6v) of a single participant with ALS. Neural activity was recorded from two 64-channel arrays (ventral and dorsal 6v), with spike band power and threshold crossings extracted to yield 256 features, while the participant attempted to speak 10,880 prompted sentences. All decoding uses z-scored neural activity during the "go" phase, provided in 20 ms time bins (50 Hz). [**willett2023high**]

Preprint. Under review.

1.3 Evaluation

We evaluate model performance using Phoneme Error Rate (PER), defined as $PER = \frac{(S+D+I)}{N}$, where S, D, and I are the numbers of substitutions, deletions, and insertions, and $N$is the total number of reference phonemes. Lower PER indicates more accurate phoneme decoding. While Word Error Rate (WER) can also be computed by mapping phonemes to words, this project focuses on improving PER on the validation set. For our project, PER is often alternatively referred to as CER.

2 Methods

2.1 Data Augmentation

We regularize our GRU-based neural decoder using two complementary data augmentation techniques on the neural speech dataset: additive Gaussian noise and time masking. Gaussian noise is added to the 20 ms–binned neural features (spike band power and threshold crossings) on each trial, mimicking variability in signal-to-noise ratio and electrode drift and encouraging the model to learn noise-robust representations []. Time masking, inspired by SpecAugment, then randomly masks contiguous blocks of time steps by replacing their features with a constant value, forcing the decoder to rely on broader temporal context rather than any single time bin. Recent work on intracortical speech BCIs and Brain-to-Text benchmarks reports similar gains when combining Gaussian noise and time masking on neural features [2], and in our experiments, this combination improves robustness to noisy or partially missing neural signals.

2.2 Time Masking

The default setup applied only Gaussian noise as data augmentation, so we implemented a new module, AugWithTimeMasking, which combines Gaussian noise and time masking, controlled by a masking fraction and a maximum number of masks. We tested three configurations—(0.10, 2), (0.15, 3), and (0.20, 4)—and found that (0.20, 4) produced the lowest CER.

2.3 Gradient Clipping

We apply gradient clipping during training to stabilize optimization of our neural decoder on the Brain-to-Text dataset. By limiting the norm of the backpropagated gradients when they become too large, we prevent exploding gradients, a known issue in RNNs and sequence models that can lead to unstable updates and divergence [**pascanu2013difficulty**]. Gradient clipping is standard in end-to-end speech recognition and CTC-based models [**qin2019new**], and in our experiments, norm-based clipping across all trainable parameters improves overall training stability.

2.4 Early Stopping

We use early stopping as a regularization strategy to prevent overfitting when training our neural decoder on the Brain-to-Text dataset. Early stopping monitors performance on a held-out validation set and halts training once the validation error (e.g., PER/CER or validation loss) stops improving for a fixed patience window, keeping the best-performing checkpoint [1].

2.5 Encoder-Side Modifications

2.5.1 Light weight Transformer Layer Normalization

We explored several modifications on the encoder-side to improve the stability and robustness of the baseline GRU model, with all experiments conducted independently from other components of the project. First, we inserted a LayerNorm module before the GRU to reduce dependence on the baseline's day-specific linear layer and to stabilize input distributions across trials. We also tested adding a lightweight Transformer encoder block (hidden size 256, 4 heads) before the GRU to enhance local temporal modeling while preserving causal constraints. All variants were trained using the same batch count, learning rate, optimizer, and validation protocol as the baseline to ensure that observed differences were attributable to architectural or augmentation changes rather than training conditions.

2.5.2 Hierarchical VAE front end replacing Per day input layer

We tested various potential optimizations which included switching the input day layer into a variational autoencoder (VAE). In the original implementation, we have an input layer per day that only consists of 65,792 (256 * 256 + 256) parameters– around 1,579,008 for the 24 days. We hoped that the VAE could be smaller and better capture similarities between days. Given data augmentation can also shift the per-day distribution we also experimented turning these off could improve encoding performance. In this segment, we also compared one-hot encoding to using the day number as an input feature.

2.6 Basic Optimizations

2.6.1 Cosine Annealing Learning Rate Scheduler

We transitioned to the lightning hydra (LH) template for easier testing and debugging; we decided to test alternative ways of optimization. With the migration to the new template, we have to lower the learning rate (LR) from 0.02 to a lower value of 0.001. We wanted to see whether maintaining a higher learning than that of linear learning rate scheduler would help performance We began with running 50 epochs over the data set (this is equivalent to increasing nBatch in the original model).

2.6.2 Increasing L2 decay Replacing Adam Optimizer with AdamW

To prevent overfitting, we decided to increase L2 deacy to more effectively penalize larger weights in the model. However, because of Adam approach to direct addition of L2 decay term to the gradient is less than ideal, we also migrated over to AdamW which have the weight decay decoupled.

2.6.3 Layer Normalization in GRU

Inspired by the TA during discussion section, we decided to also experiment with layer normalization to see if stabilizing the activation would help the performance of the GRU.

2.7 Decoder-Side Modifications

We also explored several decoder-side stages to improve the quality of the decoded phoneme sequence from the GRU and add language capabilities. First, a variety of (relatively) small, pre-trained LLMs were prompted to output a transcription. To preserve the streaming nature of the task, the decoding of short phrases was investigated. Examples of noisy phoneme sequences were saved and reused across the evaluation of different LLMs. Thinking time was enabled for models that support extended thinking. An example prompt for the LLMs is included in the appendix. Secondly, the pre-trained ASR model nvidia/parakeet-tdt-0.6b-v2 was tested. A GRU was pre-trained on the dataset. The GRU output was fed into a 3-layer transformer encoder, replacing the ASR model's acoustic encoder. The final word was converted back to phonemes. The combined GRU+ASR model was trained with both the ASR model frozen and in a fine-tune setting. Due to memory constraints, a 512-unit GRU was used. We finally explore integrating a purpose-trained phoneme denoising model. This phoneme model is a transformer decoder-encoder 128-length sequence-to-sequence (seq2seq) model that is trained on 5 million publicly available sentence examples from Project Gutenberg. Sentences were converted to sequences of phonemes using the CMU Pronouncing Dictionary. The dataset was augmented with random phoneme additions, deletions, and substitutions with (0.05, 0.05, 0.1) probability. Of substitutions, those within confusable phoneme groups had 0.8 probability. Pronunciation variations were also included to allow the model to reduce the impact of regional accent. The seq2seq model was tested with both a frozen pre-trained 512-unit GRU and the same pre-trained neural decoding stage with fine-tuning.

3 Results

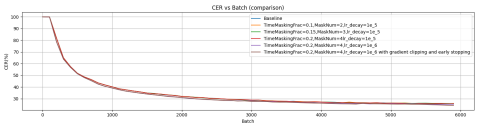

Figure 1: CER vs. 6000 Batches Running

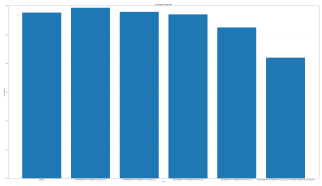

Figure 2: Final CER Value Comparison

Table 1: Validation PER for encoder-side model variations. None of the tested modifications consistently outperformed the baseline.

Model Variant	**PER**
Baseline GRU (provided)	0.223
LayerNorm before GRU	0.217–0.228
Transformer encoder (1 layer)	0.224–0.229
Higher white noise	0.225–0.231
DC offset noise	0.226–0.232
Feature masking (10–30%)	0.221–0.230

Table 2: Validation CER for the models with different optimization on LH template

Model Variant	**CER**
Adam + Linear LR Sch.	0.327
Adam + Cosine LR Sch.	0.281
AdamW + Cosine LR Sch. + Low L2	0.240
AdamW + Cosine LR Sch. + Low L2 + Layer Norm in GRU	0.223
AdamW + Cosine LR Sch. + Low L2 + Layer Norm in GRU (tuned)	0.202

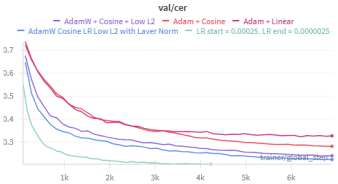

Figure 3: CER of LH Baseline Model with Different Optimization

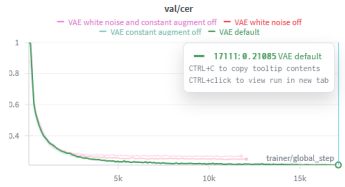

Figure 4: CER of default embedding with different conditions

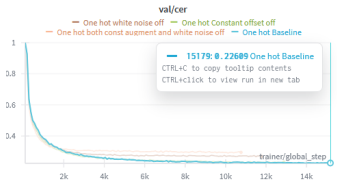

Figure 5: CER of one-hot encoding with different conditions

Table 3: Validation CER for the VAE models with different settings for whiteNoiseSTD and constantOffSet

Model Variant	CER
VAE default embedding	0.211
VAE default embedding + constant augment off	0.217
VAE default embedding + white noise off	0.249
VAE default embedding + constant augment off + white noise off	0.268
VAE one hot encoding	0.226
VAE one hot encoding + constant augment off	0.232
VAE one hot encoding + white noise off	0.269
VAE one hot encoding + constant augment off + white noise off	0.295

plain text	shoo you don't say so
target phoneme sequence	SH UW <sil> Y UW <sil> D OW N T <sil> S EY <sil> S OW
noisy phoneme sequence	SH UW <sil> UW <sil> D N T <sil> S EY UH S M OW
google/gemma-3-270m	[No response]
google/gemma-3-1b	show them a house
google/gemma-3-4b	show him I suppose
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	[Thought for 16.7 seconds] shu dent see you saw
openai/gpt-oss-20b	[Thought for 43.0 seconds] shoo you don't say more
PS2S-7m	Y UW <sil> D UW <sil> N AA T <sil> S EY <sil> S OW
PS2S-7m (converted)	you do not say so
1024-unit Diphone GRU+PS2S-7m CER	0.2500

Table 4: Example model outputs for LLMs and phoneme sequence-to-sequence (PS2S) model stages

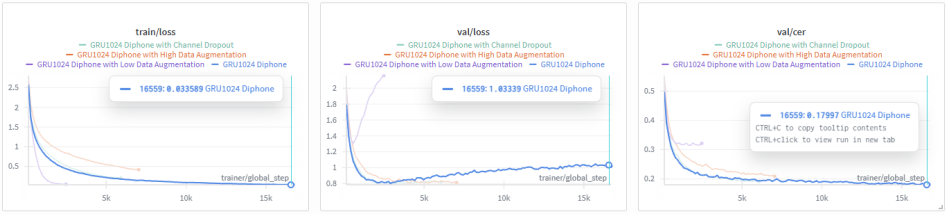

Figure 6: Training and validation statistics of GRU with AdamW + Cosine LR Sch. + Low L2 + Layer Norm in GRU + diphone encoding

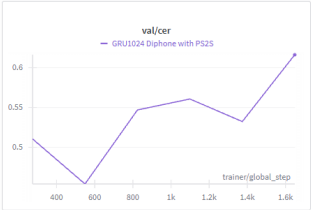

Figure 7: Validation CER of GRU with diphone encoding and PS2S denoising in fine-tuning mode

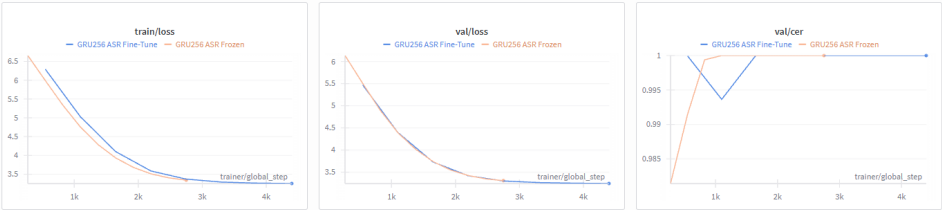

Figure 8: Training and validation statistics of GRU with ASR denoising

4 Discussion

4.1 Functionality of Time Masking

Time masking randomly zeros out contiguous spans along the time axis of the input spectrogram, hiding short temporal segments from the model during training. In end-to-end speech recognition, it is widely used as a regularizer and data augmentation technique, making models less sensitive to local distortions and more robust to temporal variability in the input signal [**park2019specaugment**]. For our GRU-based model on the neural speech dataset, time masking encourages the network to decode characters and words even when parts of the neural feature sequence are missing or unreliable, reducing overfitting and promoting more abstract, generalizable temporal representations [3]. When combined with Gaussian noise—which perturbs feature amplitudes and mimics continuous measurement noise—time masking's simulated dropouts help the GRU learn to handle both noisy amplitudes and missing frames, making the decoder better matched to real neural recording imperfections and contributing to the observed reduction in CER.

4.2 Functionality of Gradient Clipping

Gradient clipping is used in our GRU-based neural speech decoding model to keep training stable, especially for long and variable-length sequences in the neural speech dataset. Concretely, we cap the gradient norm at 5 during backpropagation: if the norm exceeds this threshold, it is rescaled back to 5 or below. This prevents very large gradients from causing extreme parameter updates, which can lead to numerical issues such as NaN/Inf values and abrupt divergence of the loss [**pascanu2013difficulty**]. By keeping gradients bounded, we can safely train on longer sequences without a few high-variance or artifact-contaminated segments dominating the update and destabilizing optimization. It also gives us more freedom to choose the learning rate, since each update is explicitly controlled; in general, controlled gradient norms combined with moderately larger learning rates have been shown to yield faster and smoother convergence for recurrent models [**pmlr-v28-pascanu13**]. Although we did not perform an extensive study of more aggressive learning-rate settings due to time constraints, our experiments indicate that enabling gradient clipping leads to noticeably more stable training curves and a more reliable reduction in character error rate (CER), particularly in later stages of training.

4.3 Encoder-Side Modifications

4.3.1 Layer Normalization

The modifications we implemented on the encoder side yielded minimal improvements across our experiments. Stronger noise augmentations and feature masking successfully delayed overfitting but did not improve validation performance. Across all experiments, PER converged close to the baseline, indicating that encoder-side modifications alone were insufficient to yield consistent gains. The limited impact of LayerNorm indicates that the baseline's day-specific linear transformation already provides the necessary normalization to account for cross-day variability, rendering additional normalization unnecessary. The inconsistent performance of the Transformer encoder indicates that attention-based temporal modeling may require substantially more extensive tuning or computational resources to outperform the simpler GRU-based architecture in a causal setting. The augmentation experiments indicate that the baseline model is already effectively optimized: the introduction of white noise or DC offset noise delayed overfitting, but did not improve validation PER, and electrode dropout did not yield significant advantages, despite theoretically enhancing the model's robustness.

These findings suggest that the model's inaccuracies are not mainly attributable to slight variations in individual features but are predominantly due to more substantial structural shortcomings within the GRU encoder. These adverse outcomes provide valuable insights: modest modifications to the encoder or solely implementing more extensive augmentation are unlikely to yield enhancements surpassing the well-optimized baseline. To achieve substantial progress, it may be necessary to implement more significant architectural modifications or develop innovative approaches for the decoder's operation.

4.3.2 Potential of Hierarchal VAE encoding layer

The default embedding out performs one hot encoding and keeps both parameters (whiteNoiseSTD and constantOffSet) appears to be the best option. This landed us around a CER near 0.21085 at the end of the training cycle. The standard baseline established around 0.22 is still a minimal improvement. Although this approach only offered minimal improvement to the baseline model (CER: 0.21) we are able to achieve this through 10 million fewer parameters within the GRU model. In addition, this implementation allowed for more flexibility in expanding the number of different days in the data set we input. Due to constraints, we are not able to fully perform the following idea, but we would also like to propose that others try first training the VAE and GRU on data from one specific day and then freeze the GRU section so that the VAE has to learn to transition data from all 24 days into the similar form of data from that one specific day. Training the model with these preconditions may bring additional improvements.

4.4 Baseline Optimization

For the LH models we only conducted 50 epochs 3 (best one converged earlier thus less) thus for some the PER maybe higher than the base line model 2 These are not final converging values but for a quick trial to test if optimization is effective or not. In our case we observe that when applying cosine annealing LR scheduler, with AdamW optimizer, low L2 decay, and Layer normalization with a starting LR of 0.0002 and ending LR of 0.0000025 (batch size: 96) we see the model converges around a CER of 0.202 which is an improvement compared to the base line cer of 0.22. This tells us having and maintaining a slightly higher LR, and punishing overfitting help the model to learn.

4.5 Diphoneme encoding and decoding

Switching from phoneme encoding and decoding to diphones, which increases expressible tokens from 40 to 1600, also improves CER of the model by about 0.02, as shown by Figure 6 and comparing to the highest quality model in 2. We suggest that the improved performance due to diphoneme encoding and decoding is due to the ability for the model to express rapid transitions in phonemes which occur inside the kernel length of the GRU.

4.6 Discussion of Decoder-Side Experiments

Qualitatively, LLMs tend to perform poorly at direct phoneme-to-word conversion in a streaming setup. Table 4 shows that model size must explode relative to the base 1024-unit GRU's 60 million parameter count for only meager increases in performance. Thinking models such as OpenAI's gpt-oss-20b are accurate but are unable to meet low-latency requirements. A purpose-built solution, such as the phoneme sequence-to-sequence denoising model, is shown to be lightweight but still yields cohesive output. If fed the output of the high-quality pretrained GRU model (AdamW + Cosine LR Sch. + Low L2 + Layer Norm in GRU), the overall model accuracy lowers, however. This break in cohesion and CER implies that CER may not fully capture the complexity of a neural-activity-to-speech problem. ASR models show promise as they integrate language capabilities with streaming capabilities, but the implementation tested in 8 was unable to provide intelligible output. We suspect that an approach that converts noisy phoneme sequences into audio using text-to-speech software before being fed into the ASR model would fare better, but this setup is not only clunky but counterproductive if the goal is a lightweight neural-activity-to-speech model.

References

[1] F. R. Willett, E. M. Kunz, C. Fan, D. T. Avansino, G. H. Wilson, E. Y. Choi, F. Kamdar, M. F. Glasser, L. R. Hochberg, S. Druckmann, et al. *A high-performance speech neuroprosthesis*. Nature, 620(7976):1031–1036, 2023.

[2] E. Lashgari, D. Liang, and U. Maoz. *Data augmentation for deep-learning-based electroencephalography*. Journal of Neuroscience Methods, 346:108885, 2020.

[3] S. Wen, A. Yin, T. Furlanello, M. G. Perich, L. E. Miller, and L. Itti. *Rapid adaptation of brain–computer interfaces to new neuronal ensembles or participants via generative modelling*. Nature Biomedical Engineering, 7(4):546–558, 2023.

[4] Y. Zhang, L. He, C. Fan, T. Liu, H. Yu, T. Le, J. Li, S. Linderman, L. Duncker, F. R. Willett, N. Mesgarani, and L. Paninski. *Decoding inner speech with an end-to-end brain-to-text neural interface*. arXiv preprint arXiv:2511.21740, 2025. doi:10.48550/arXiv.2511.21740.

[5] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le. *SpecAugment: A simple data augmentation method for automatic speech recognition*. arXiv preprint arXiv:1904.08779, 2019.

[6] C.-X. Qin, W.-L. Zhang, and D. Qu. *A new joint CTC-attention-based speech recognition model with multi-level multi-head attention*. EURASIP Journal on Audio, Speech, and Music Processing, 2019(1):18, 2019.

[7] B. Accou, J. Vanthornhout, H. Van hamme, and T. Francart. *Decoding of the speech envelope from EEG using the VLAAI deep neural network*. Scientific Reports, 13(1):812, 2023.

[8] P. Bahar, A. Zeyer, R. Schlüter, and H. Ney. *On using SpecAugment for end-to-end speech translation*. arXiv preprint arXiv:1911.08876, 2019.

[9] R. Pascanu, T. Mikolov, and Y. Bengio. *On the difficulty of training recurrent neural networks*. In Proceedings of the 30th International Conference on Machine Learning (ICML), pages 1310–1318, 2013.

A Appendix

A.1 Example LLM prompt

Your task is to perform automatic speech recognition. You are given a candidate of an unknown transcription. Your job is to come up with a transcription that is more accurate, relying on context and the norms of spoken English. Some phonemes in the phoneme sequences might have been added, removed, or replaced. Avoid repetitive or nonsensical phrases. In cases of ambiguity, output the phrase that is most coherent and contextually sound. Respond with your final transcription only, without any introductory text.

SH UW <sil> UW <sil> D N T <sil> S EY UH S M OW

A.2 Code availability

The code is available at the following public GitHub repository: https://github.com/johnnyou1873/c143a-project Datasets and final trained weights are available from the authors upon request.